

INSTITUTO FEDERAL
Piauí - Campus Floriano

Polimorfismo em Java

Arquitetura, Flexibilidade e o Poder de Múltiplas Formas

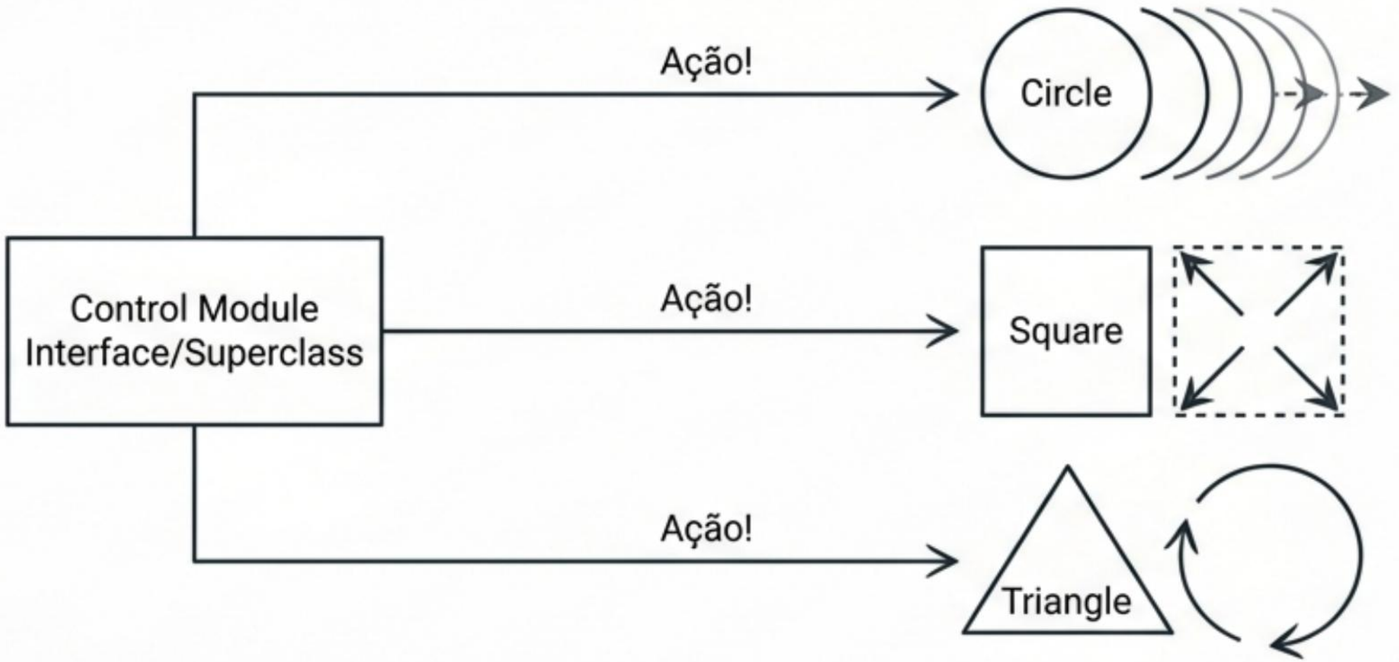
Prof. Ronaldo Borges

NotebookLM

O Princípio Fundamental

Uma interface, muitas formas.

O polimorfismo permite que objetos diferentes respondam à mesma chamada de método, cada um à sua própria maneira. O sistema trata as generalidades; o objeto trata as especificidades.



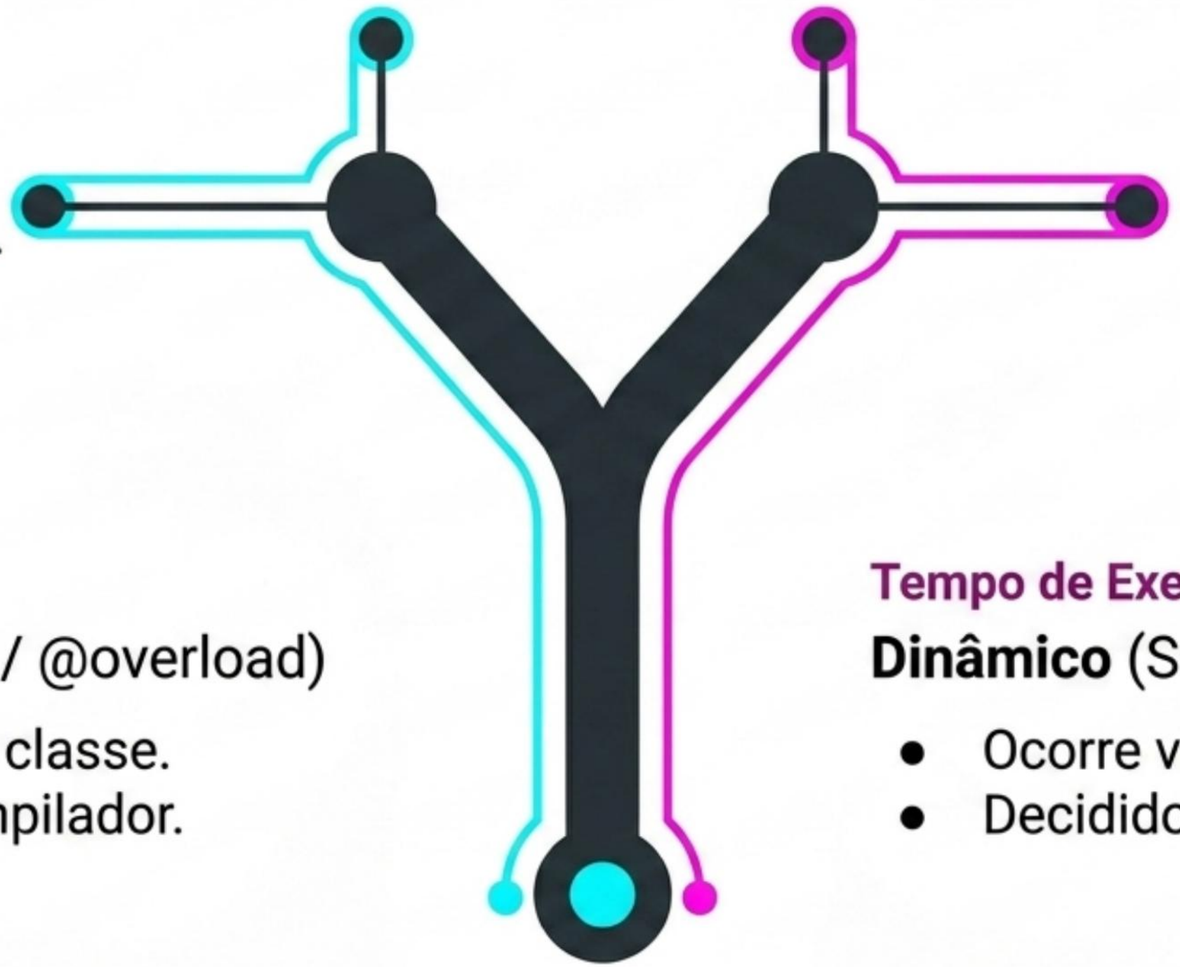
ronaldo.pb@ifpi.edu.br

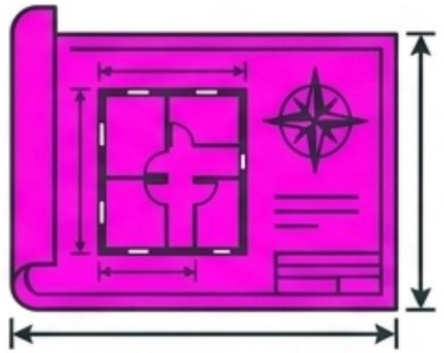
NotebookLM

As Duas Faces do Polimorfismo



Faca Suíça





Planta Baixa

Tempo de Compilação

Estático (Sobrecarga / @overload)

- Ocorre na mesma classe.
- Decidido pelo compilador.

Tempo de Execução

Dinâmico (Sobrescrita / @override)

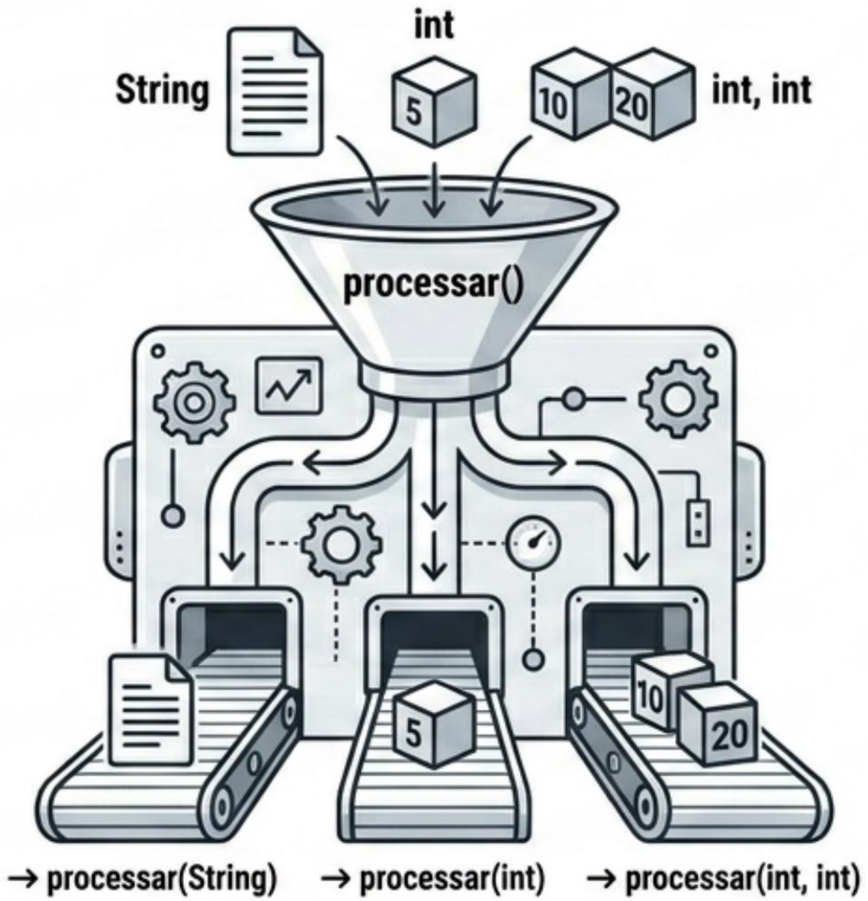
- Ocorre via herança.
- Decidido pela JVM em execução.

ronaldo.pb@ifpi.edu.br

NotebookLM

Sobrecarga de Métodos (@overload)

Múltiplos métodos com o mesmo nome, mas assinaturas diferentes (número, tipo ou ordem dos parâmetros). O compilador roteia a chamada com base nos argumentos fornecidos.

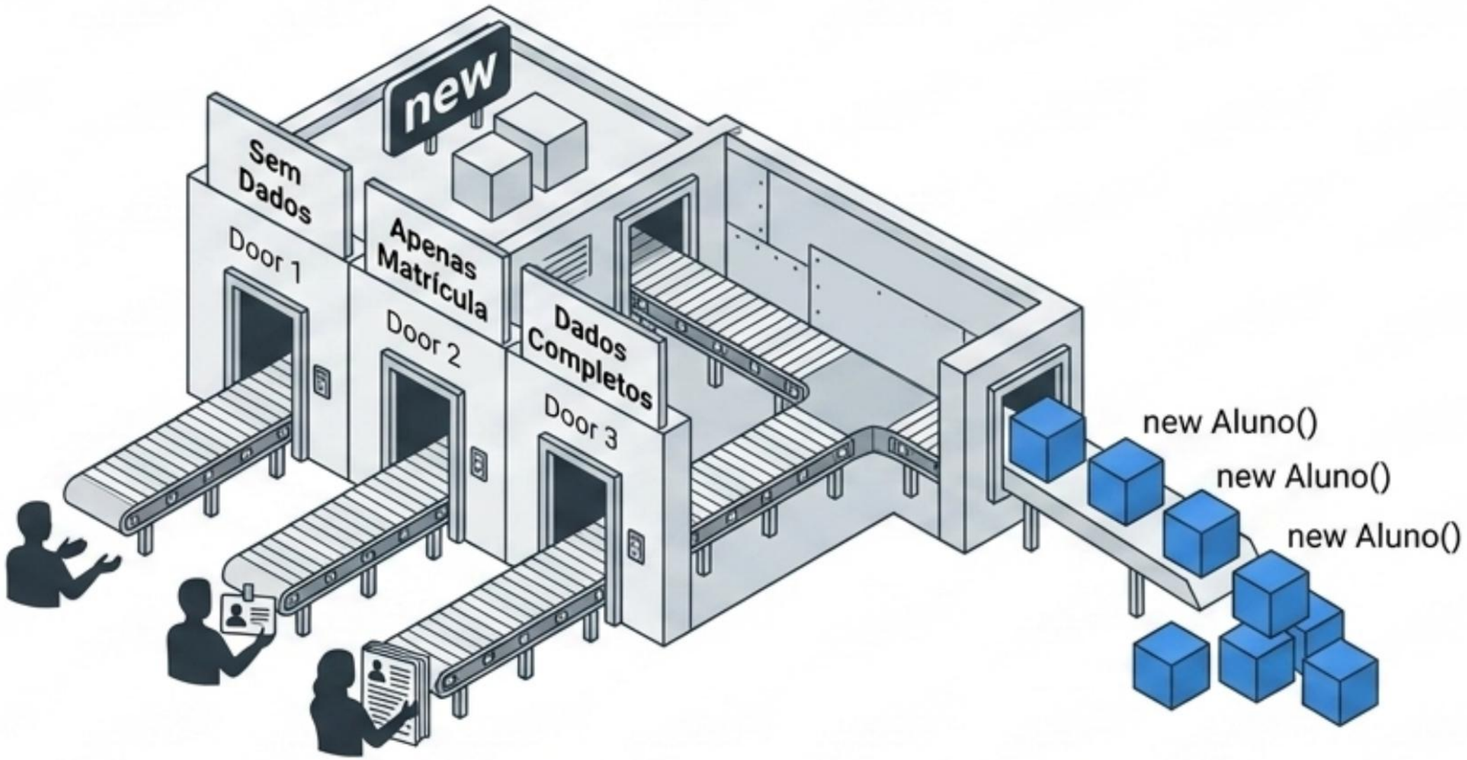



```java
public void processar(String texto) { ... }
public void processar(int numero) { ... }
public void processar(int a, int b) { ... }
```

ronaldo.pb@ifpi.edu.br
NotebookLM

A Fábrica Flexível: Sobrecarga de Construtores

Construtores também podem ser sobrecarregados. Isso oferece flexibilidade à arquitetura, permitindo instanciar objetos com diferentes níveis de informação inicial.



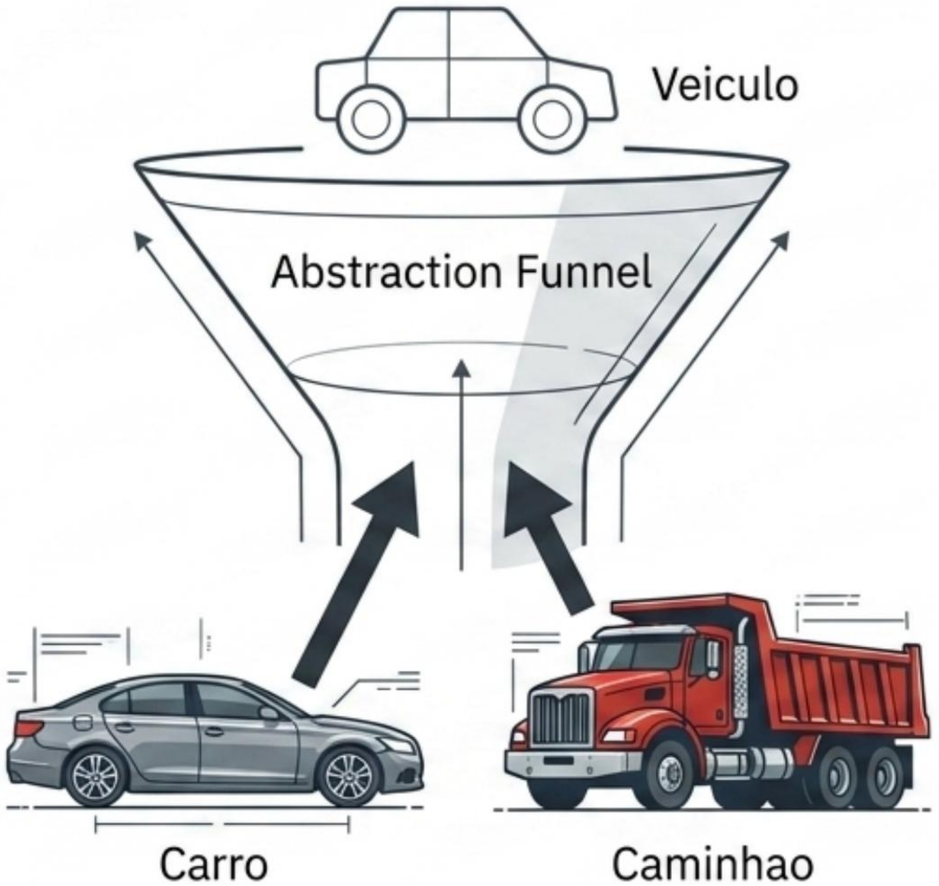

```
Aluno aluno1 = new Aluno();
Aluno aluno2 = new Aluno("2023101");
Aluno aluno3 = new Aluno("2023101", "Maria");
```

ronaldo.pb@ifpi.edu.br | (

NotebookLM

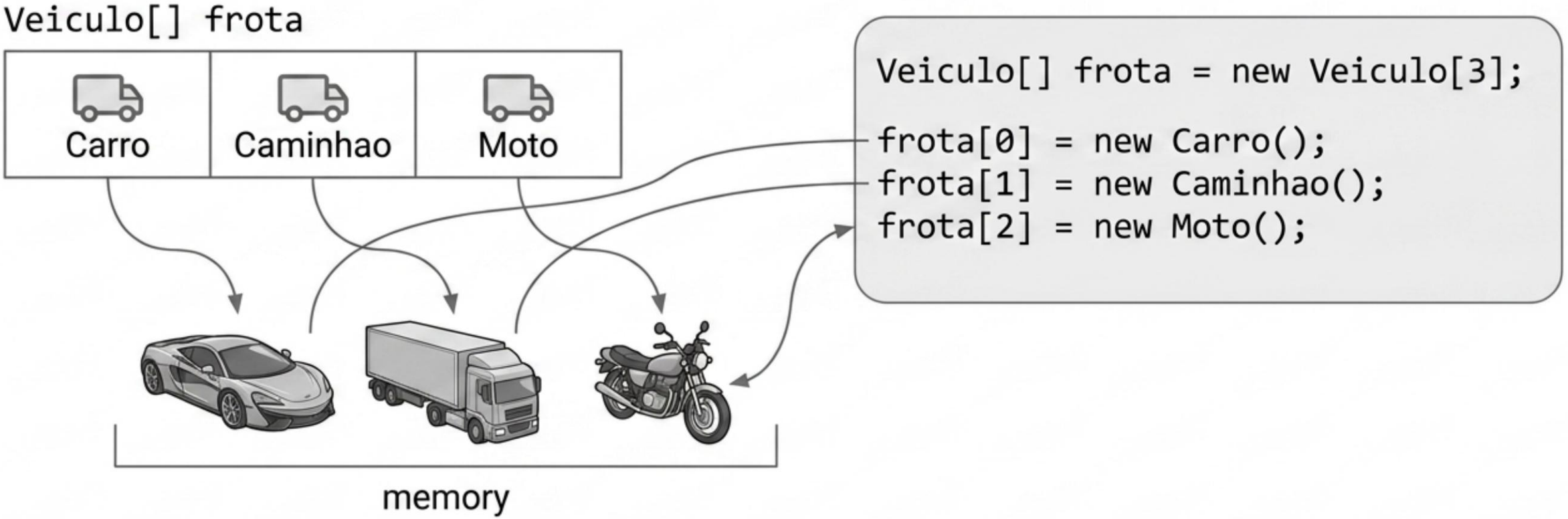
Generalização: O Relacionamento "É-Um"

Um objeto de uma subclasse (Carro) é um objeto da superclasse (Veículo). A generalização permite tratar objetos específicos como genéricos, ignorando os detalhes que não importam no momento.



Programando no Geral

A grande vantagem da generalização é a criação de coleções heterogêneas.
O sistema enxerga apenas Veiculos, simplificando a lógica de controle.

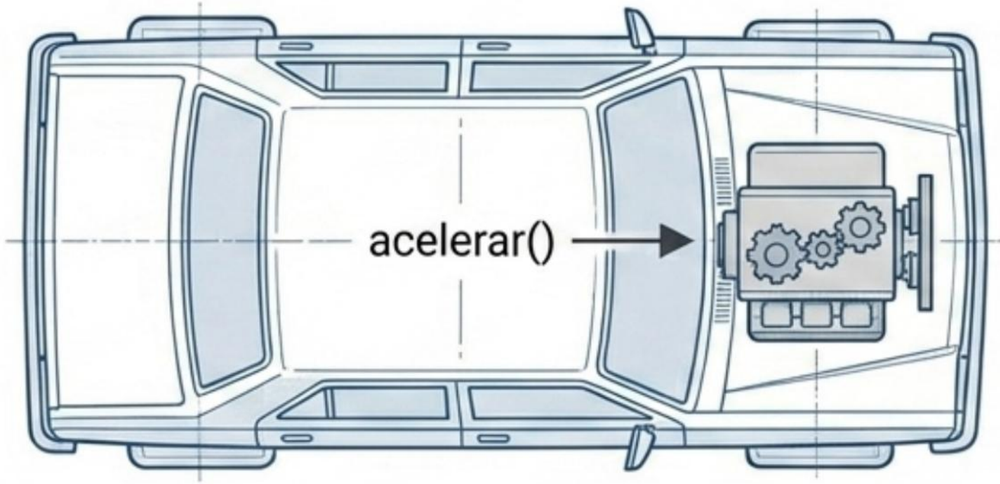


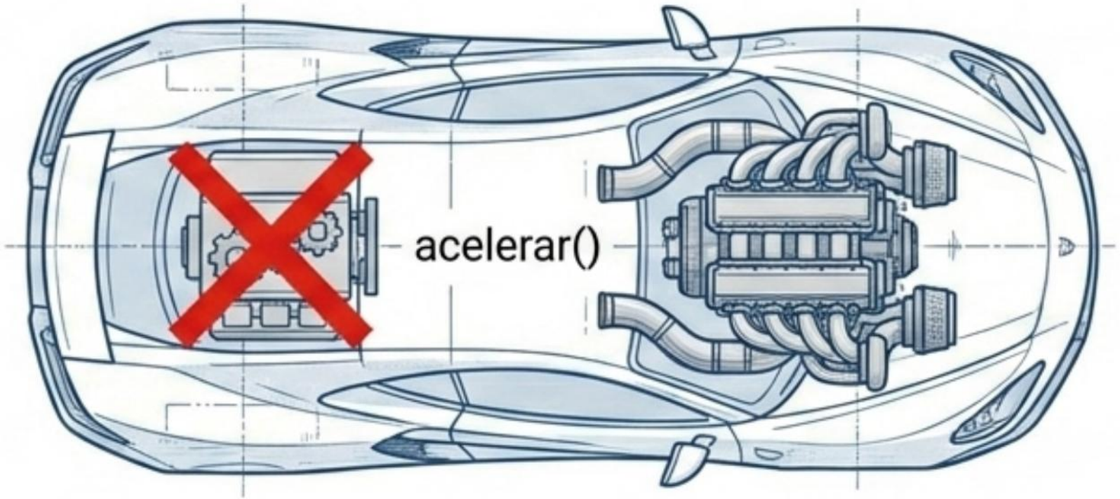
ronaldo.pb@ifpi.edu.br | (, , , n ,)

NotebookLM

Sobrescrita de Métodos (@override)

Ocorre quando uma subclasse fornece sua própria implementação de um método herdado. A mesma mensagem (acelerar) gera comportamentos distintos baseados na classe real do objeto.






```
// Na Superclasse Veiculo:
public void acelerar() { velocidade += 10; }

// Na Subclasse Carro:
@Override
public void acelerar() { velocidade += 50; }
```

ronaldo.pb@ifpi.edu.br | (, , , n ,)

NotebookLM

A Rede de Segurança: Porque usar @Override?

A anotação **@Override** é opcional, mas vital. Ela obriga o compilador a verificar se você está realmente sobrescrevendo um método da superclasse. Um simples erro de digitação (ex: **tostring** em vez de **toString**) sem a anotação cria um método novo silenciosamente, gerando bugs difíceis de rastrear.

Sem Anotação



Com @Override



ronaldo.pb@ifpi.edu.br
NotebookLM

Resolução Dinâmica (Late Binding)

Quando um método sobrescrito é chamado por uma variável de superclasse, o Java decide qual versão executar com base no tipo real do objeto instanciado na memória em tempo de execução, e não no tipo da variável de referência.



NotebookLM

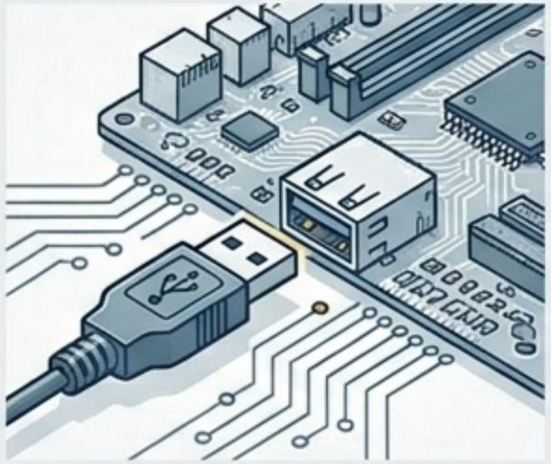
A Matriz Definitiva: Sobrecarga vs. Sobrescrita

Característica	Sobrecarga (Overload)	Sobrescrita (Override)
Localização	Na mesma classe	Em uma subclasse (Hierarquia)
Assinatura	**DEVE** ser diferente (parâmetros)	**DEVE** ser idêntica
Tipo de Retorno	Pode mudar livremente	Deve ser o mesmo (ou covariante)
Resolução	Tempo de Compilação (Estática)	Tempo de Execução (Dinâmica)

ronaldo.pb@ifpi.edu.br |
NotebookLM

Vantagens da Arquitetura Polimórfica

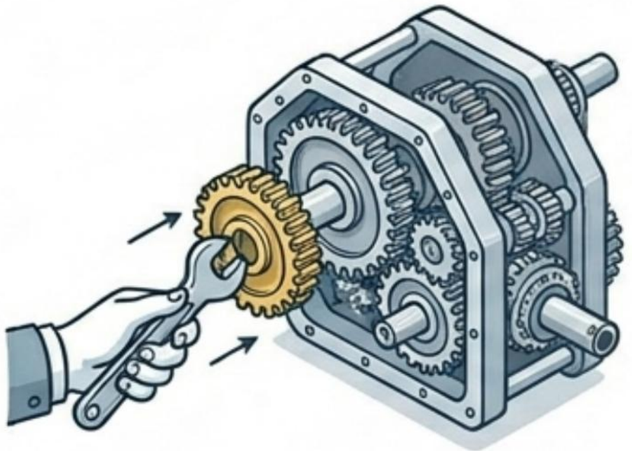
Extensibilidade



Extensibilidade

Novos tipos podem ser adicionados ao sistema com pouca ou nenhuma modificação no código existente.

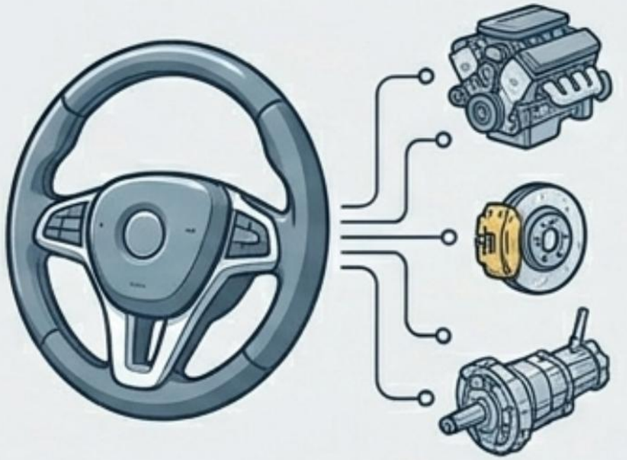
Manutenibilidade



Manutenibilidade

O código que invoca o comportamento é independente dos tipos específicos de objetos.

Simplicidade



Simplicidade

Reduz a complexidade ao permitir que uma mesma interface seja usada para especificar uma classe geral de ações.

ronaldo.pb@ifpi.edu.br |

NotebookLM

Pontos de Atenção e Armadilhas



A Falsa Sobrecarga

Alterar apenas o tipo de retorno não é suficiente para caracterizar uma sobrecarga. O compilador gerará um erro.



O Perigo do Object

Toda classe herda de Object. Fazer casting indiscriminado quebrando o polimorfismo gera erros de ClassCastException em tempo de execução.



Herança vs. Composição

Não force uma relação “É-Um” apenas para reaproveitar código. O polimorfismo só faz sentido se a subclasse puder legitimamente substituir a superclasse.

NotebookLM

Referências Bibliográficas

BEZERRA, Eduardo. Princípios de análise e projeto de sistemas com UML. 3. ed. Rio de Janeiro: Elsevier, 2015.

DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

SCHILDT, Herbert. Java para iniciantes: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015.

TERUEL, Evandro Carlos. Programação Orientada a Objetos com Java: sem mistérios. São Paulo: Editora, 2016.

ronaldo.pb@ifpi.edu.br | (, , p.)

NotebookLM